PYTHON ACTIVITY-BASED LEARNING

Complete Report – 10 Selected Case Studies

Student Name: Jasmitha

Roll Number: 26001057

Branch: CSM

Section: 4

Course: Python Programming

Institution: Dr. RVR NRI Institute of Technology

This report is organised according to the ten sections required in the college ABL circular. Each case includes the problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, an output screenshot space, and conclusion.

Selected Case Studies

Case 1: University Scholarship Allocation

Case 4: Employee Payroll and Deduction Analysis

Case 13: ATM Transaction Sequence Simulation

Case 42: Student Marks Matrix Analysis

Case 53: Food Waste Monitoring System

Case 58: Library Borrowing and Fine Reconciliation

Case 66: Student Academic Risk Classification

Case 71: Integrated University Fee and Eligibility Audit

Case 84: Environmental Monitoring Data Analyzer

Case 100: Comprehensive Smart Campus Management Simulation

Note: The college circular asks students to insert their own screenshots after running the programs. The screenshot pages below use the terminal images supplied in this chat where available. For Case 13, the supplied run contains invalid menu selections; the corrected input sequence is explained in that section. For Case 53, no matching output screenshot was supplied, so a blank screenshot area is provided.

Case 1: University Scholarship Allocation

1. Problem Statement

Develop a Python program to decide whether a student qualifies for a university scholarship using subject marks, aggregate, attendance and annual family income.

2. Objectives

Accept the student's name and five subject marks; calculate the average; check the scholarship conditions; print a clear eligibility report.

3. Assumptions

Five subjects are considered. Aggregate must be at least 80%; each subject mark must be at least 70%; attendance must be at least 75%; annual family income must be at most Rs. 2,50,000. These are the rules used by the supplied program.

4. Algorithm

1. Start.

2. Read student name and five subject marks.

3. Calculate aggregate = sum of marks / 5.

4. Read attendance and annual family income.

5. Check aggregate >= 80, minimum subject mark >= 70, attendance >= 75 and income <= 250000.

6. Display eligible only if all conditions are true; otherwise display not eligible.

7. Stop.

5. Flowchart

Start → Input name, five marks, attendance and income → Calculate aggregate → Check all conditions → Display result → Stop.

6. Pseudocode

```
INPUT name and five subject marks
aggregate = SUM(marks) / 5
INPUT attendance, income
IF aggregate >= 80 AND MIN(marks) >= 70 AND attendance >= 75 AND income <= 250000
    PRINT "Eligible for scholarship"
ELSE
    PRINT "Not eligible for scholarship"
END IF
```

7. Python Source Code

```
name = input("Enter student name: ")
marks = []
for i in range(5):
    mark = float(input("Enter subject " + str(i + 1) + " marks: "))
    marks.append(mark)

attendance = float(input("Enter attendance percentage: "))
income = float(input("Enter annual family income: "))
aggregate = sum(marks) / len(marks)

if aggregate >= 80 and min(marks) >= 70 and attendance >= 75 and income <= 250000:
    result = "Eligible for scholarship"
else:
    result = "Not eligible for scholarship"

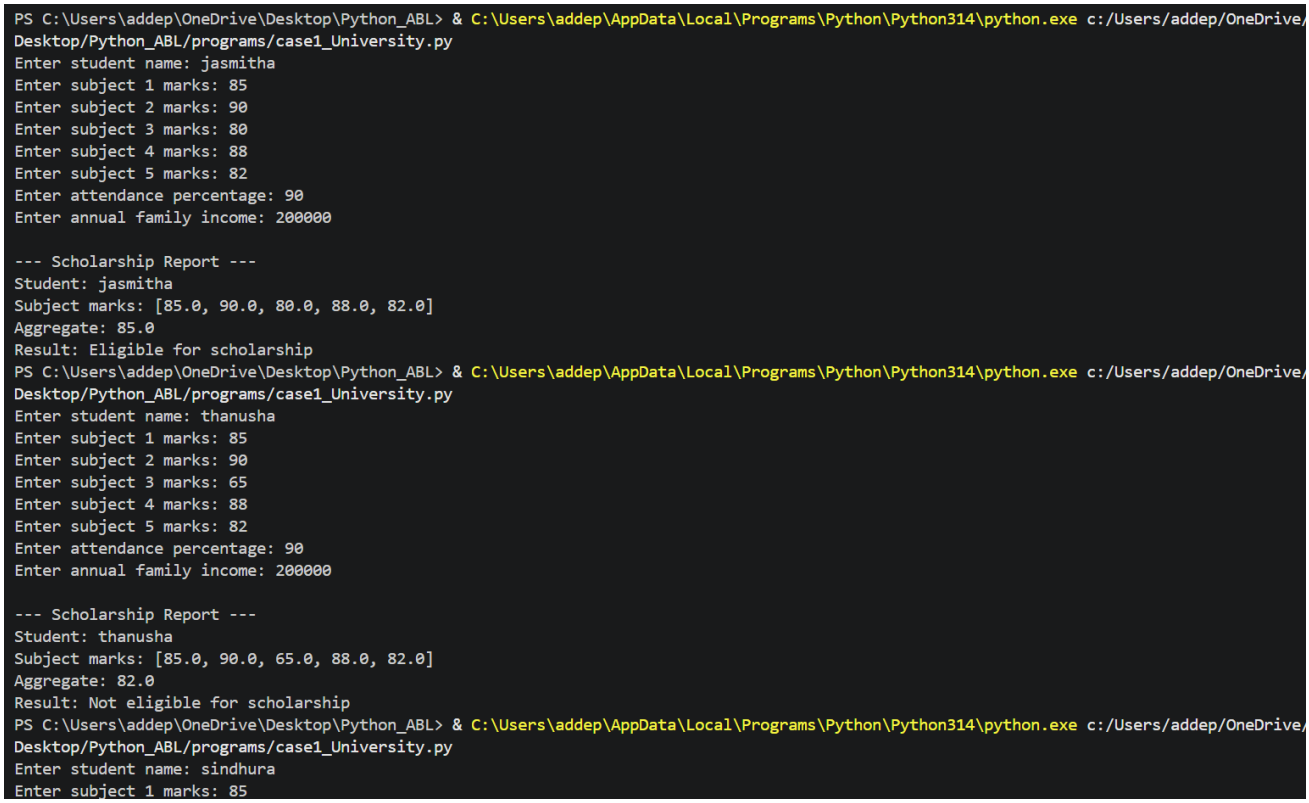
print("\n--- Scholarship Report ---")
print("Student:", name)
print("Subject marks:", marks)
print("Aggregate:", aggregate)
print("Result:", result)
```

8. Test Cases

Test Case 1 – Input: Jasmitha; marks 85,90,80,88,82; attendance 90; income 200000
Expected/observed result: Aggregate 85.0; Eligible for scholarship

Test Case 2 – Input: Thanusha; marks 85,90,65,88,82; attendance 90; income 200000
Expected/observed result: Aggregate 82.0; Not eligible (one subject is below 70)

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The University Scholarship Allocation program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Create an employee payroll program that calculates allowances, gross salary, deductions and net salary from the basic salary.

2. Objectives

Read employee name and basic salary; calculate HRA, allowance, PF and insurance; display gross salary, deductions and take-home salary.

3. Assumptions

Based on the supplied output: HRA = 20% of basic salary; allowance = 10%; PF = 5%; insurance = 2%. The percentages are assignment assumptions.

4. Algorithm

1. Start.
2. Read employee name and basic salary.
3. Calculate HRA = 20% and allowance = 10% of basic salary.
4. Calculate gross salary = basic + HRA + allowance.
5. Calculate PF = 5% and insurance = 2% of basic salary.
6. Calculate total deductions and net salary.
7. Display the payroll report.
8. Stop.

5. Flowchart

Start → Input employee and basic salary → Calculate allowances → Calculate gross salary → Calculate deductions → Calculate net salary → Display report → Stop.

6. Pseudocode

```
INPUT employee_name, basic_salary
hra = basic_salary * 0.20
allowance = basic_salary * 0.10
gross = basic_salary + hra + allowance
pf = basic_salary * 0.05
insurance = basic_salary * 0.02
deductions = pf + insurance
net_salary = gross - deductions
PRINT payroll report
```

7. Python Source Code

```
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: "))

hra = basic * 0.20
allowance = basic * 0.10
gross = basic + hra + allowance
pf = basic * 0.05
insurance = basic * 0.02
total_deductions = pf + insurance
net_salary = gross - total_deductions

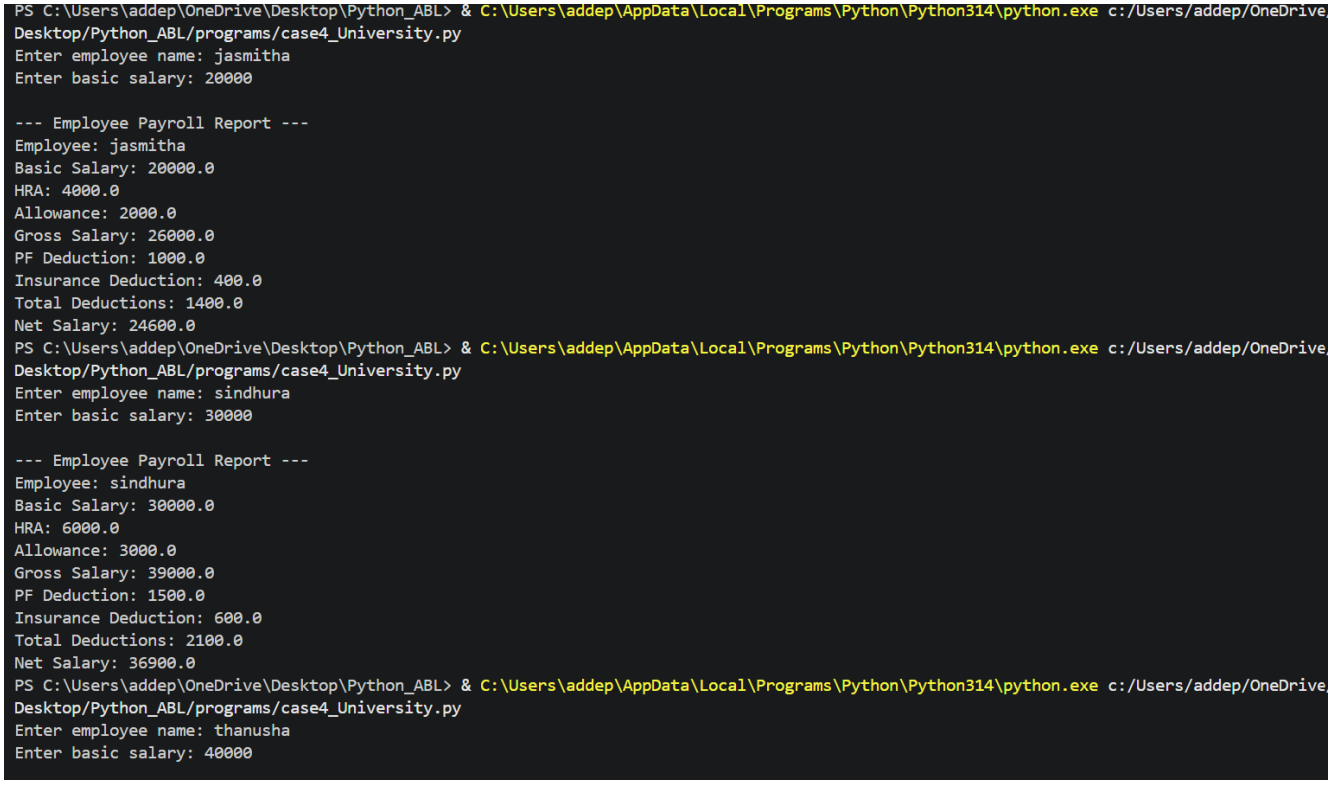
print("\n--- Employee Payroll Report ---")
print("Employee:", name)
print("Basic Salary:", basic)
print("HRA:", hra)
print("Allowance:", allowance)
print("Gross Salary:", gross)
print("PF Deduction:", pf)
print("Insurance Deduction:", insurance)
print("Total Deductions:", total_deductions)
print("Net Salary:", net_salary)
```

8. Test Cases

Test Case 1 – Input: Jasmitha; basic salary 20000
Expected/observed result: HRA 4000; allowance 2000; gross 26000; deductions 1400; net salary 24600

Test Case 2 – Input: Sindhura; basic salary 30000
Expected/observed result: HRA 6000; allowance 3000; gross 39000; deductions 2100; net salary 36900

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Employee Payroll and Deduction Analysis program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 13: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate basic ATM operations including deposits, withdrawals, balance enquiries and transaction history.

2. Objectives

Provide a menu; validate amounts; prevent withdrawal beyond available balance; record successful transactions; display final balance.

3. Assumptions

Opening balance is Rs. 10,000. Deposits and withdrawals must be positive. No additional daily withdrawal limit is applied.

4. Algorithm

1. Start with balance 10000 and an empty transaction list.
2. Display the four menu choices.
3. Read the menu choice.
4. For deposit, read a positive amount, add it to balance and record it.
5. For withdrawal, read a positive amount; withdraw only if amount <= balance and record it.
6. For balance enquiry, display current balance.
7. Repeat until Exit; print transaction history and final balance.
8. Stop.

5. Flowchart

Start → Set balance and history → Show menu → Read choice → Perform selected operation → Update balance/history → Repeat until Exit → Print history and final balance → Stop.

6. Pseudocode

```
balance = 10000
transactions = []
REPEAT
    DISPLAY menu: 1 Deposit, 2 Withdraw, 3 Balance, 4 Exit
    INPUT choice
    IF choice = 1: INPUT amount; if amount > 0, add and record deposit
    ELSE IF choice = 2: INPUT amount; if 0 < amount <= balance, subtract and record withdrawal
    ELSE IF choice = 3: DISPLAY balance
    ELSE IF choice = 4: EXIT LOOP
    ELSE: DISPLAY invalid choice
UNTIL exit
DISPLAY transaction history and final balance
```

7. Python Source Code

```
balance = 10000
transactions = []

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        amount = float(input("Enter deposit amount: "))
        if amount > 0:
            balance += amount
            transactions.append(("Deposit", amount))
            print("Deposit successful")
        else:
            print("Enter a valid amount")
    elif choice == 2:
        amount = float(input("Enter withdrawal amount: "))
        if amount <= 0:
            print("Enter a valid amount")
```

```
        elif amount <= balance:
            balance -= amount
            transactions.append(("Withdrawal", amount))
            print("Withdrawal successful")
        else:
            print("Insufficient balance")
    elif choice == 3:
        print("Current balance:", balance)
    elif choice == 4:
        break
    else:
        print("Invalid choice")

print("\n--- Transaction History ---")
for transaction in transactions:
    print(transaction[0], ":", transaction[1])
print("Final balance:", balance)
```

8. Test Cases

Test Case 1 – Input: Menu 1, amount 2000; menu 2, amount 1500; menu 3; menu 4
Expected/observed result: Deposit successful; withdrawal successful; balance/final balance Rs. 10500

Test Case 2 – Input: Menu 2, amount 12000; menu 4
Expected/observed result: Insufficient balance; final balance Rs. 10000

Test Case 3 – Input: Menu 1, amount -100; menu 4
Expected/observed result: Invalid amount; balance remains Rs. 10000

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\addep\OneDrive\Desktop\Python_ABL> & C:\Users\addep\AppData\Local\Programs\Python\Python314\python.exe c:/Users/addep/OneDrive/
Desktop/Python_ABL/programs/case13_university.py

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 10000
Invalid choice

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 12000
Invalid choice

1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 7000
Invalid choice
```

Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

Important correction: At the menu prompt, enter 1, 2, 3 or 4—not the transaction amount. For example, enter 1, then enter 2000 when asked for the deposit amount; enter 2, then enter 1500 when asked for the withdrawal amount.

10. Conclusion

The ATM Transaction Sequence Simulation program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 42: Student Marks Matrix Analysis

1. Problem Statement

Store marks of students in a nested list and calculate each student's total and average to identify the top performer.

2. Objectives

Accept marks for three subjects for each of three students; store the marks as a matrix; calculate totals and averages; find the highest total.

3. Assumptions

The sample program processes three students and three subjects (Python, Maths and English). Marks are out of 100. Student numbers are used as labels.

4. Algorithm

1. Start.
2. Create an empty marks matrix and totals list.
3. For each of three students, input Python, Maths and English marks.
4. Store each student's marks as one row.
5. Calculate each total and average.
6. Find the position of the highest total.
7. Display all reports and the top performer.
8. Stop.

5. Flowchart

Start → Create matrix → Input 3 marks per student → Calculate totals and averages → Find maximum total → Display reports/top performer → Stop.

6. Pseudocode

```
matrix = empty list
totals = empty list
FOR each of 3 students
    INPUT Python, Maths, English marks
    append the three marks as one row
    calculate row total and average
    store total
END FOR
Find index of maximum total
PRINT each student's marks, total and average
PRINT top performer
```

7. Python Source Code

```
students = []
totals = []

for i in range(3):
    print("\nEnter marks for Student", i + 1)
    row = []
    row.append(float(input("Enter Python marks: ")))
    row.append(float(input("Enter Maths marks: ")))
    row.append(float(input("Enter English marks: ")))
    students.append(row)
    totals.append(sum(row))

print("\n--- Student Marks Report ---")
for i in range(3):
    average = totals[i] / 3
    print("Student", i + 1)
    print("Marks:", students[i])
    print("Total:", totals[i])
    print("Average:", average)

top_index = totals.index(max(totals))
print("\nTop Performer: Student", top_index + 1)
```

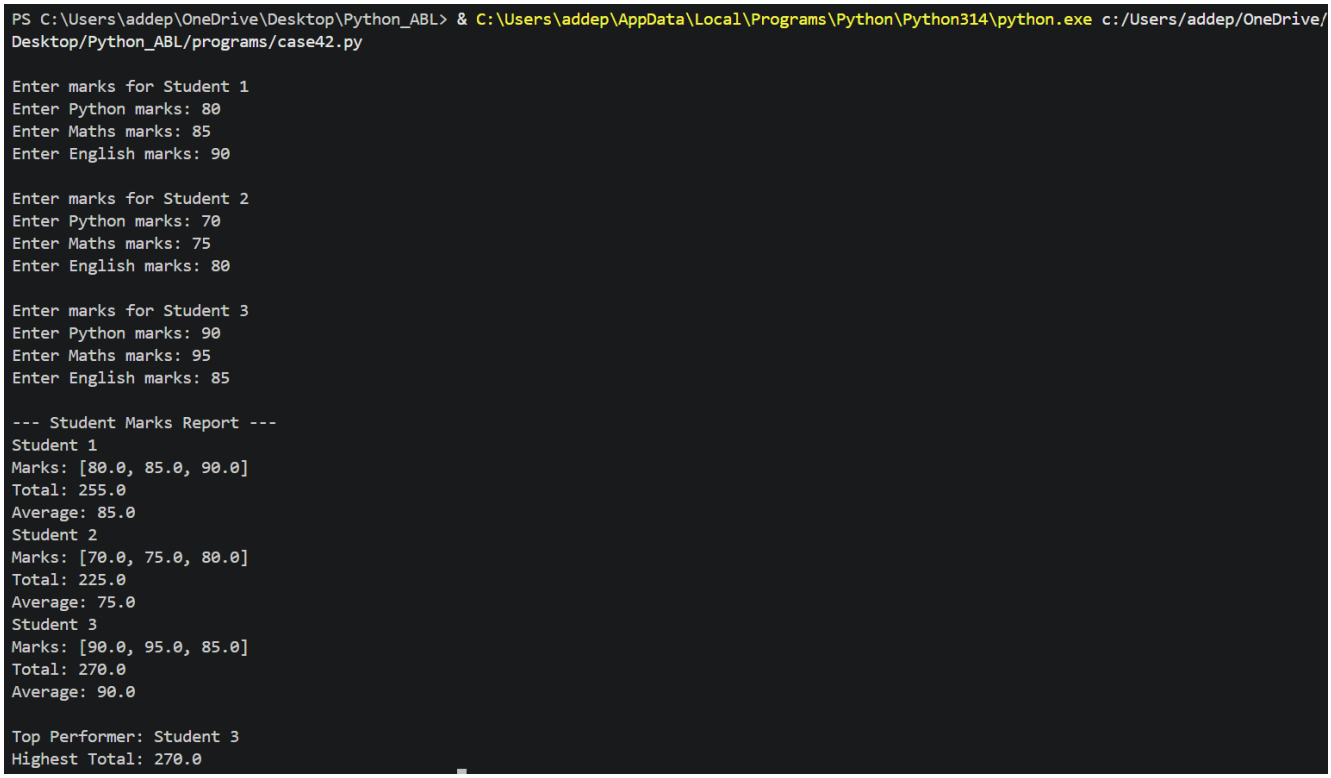
```
print("Highest Total:", max(totals))
```

8. Test Cases

Test Case 1 – Input: S1: 80,85,90; S2: 70,75,80; S3: 90,95,85
Expected/observed result: Totals 255, 225, 270; averages 85, 75, 90; Student 3 is top performer

Test Case 2 – Input: S1: 60,60,60; S2: 80,80,80; S3: 70,70,70
Expected/observed result: Totals 180, 240, 210; Student 2 is top performer

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Student Marks Matrix Analysis program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 53: Food Waste Monitoring System

1. Problem Statement

Monitor daily food waste quantities, calculate total and average waste, and identify days that exceed a chosen limit.

2. Objectives

Read daily waste quantities; store values in a list; calculate total and average waste; report normal or alert status for each day.

3. Assumptions

The provided material identifies the case title but does not specify a waste limit or fine. For this sample solution, a day is flagged when waste exceeds 20 kg. This threshold is an illustrative assumption.

4. Algorithm

1. Start.

2. Read number of monitoring days.

3. Input daily waste in kilograms and store each value in a list.

4. Calculate total and average waste.

5. For each day, check whether waste is greater than 20 kg.

6. Display Normal or Waste Alert.

7. Display total and average waste.

8. Stop.

5. Flowchart

Start → Input day count → Read and store daily waste → Calculate total/average → Check each value against 20 kg → Display status and summary → Stop.

6. Pseudocode

```
INPUT number_of_days
waste = empty list
FOR each day
    INPUT daily_waste_kg
    append daily_waste_kg to waste
END FOR
total = SUM(waste)
average = total / number_of_days
FOR each value in waste
    IF value > 20: PRINT "Waste Alert"
    ELSE: PRINT "Normal"
END FOR
PRINT total and average
```

7. Python Source Code

```
days = int(input("Enter number of days: "))
waste = []

for i in range(days):
    value = float(input("Enter food waste for Day " + str(i + 1) + " (kg): "))
    waste.append(value)

total = sum(waste)
average = total / days if days > 0 else 0

print("\n--- Food Waste Monitoring Report ---")
for i in range(days):
    status = "Waste Alert" if waste[i] > 20 else "Normal"
    print("Day", i + 1, "Waste:", waste[i], "kg")
    print("Status:", status)

print("Total Waste:", total, "kg")
print("Average Daily Waste:", round(average, 2), "kg")
```

8. Test Cases

Test Case 1 – Input: 10, 25, 15, 30, 20 kg
Expected/observed result: Total 100 kg; average 20 kg; alerts on days 2 and 4

Test Case 2 – Input: 5, 8, 12 kg
Expected/observed result: Total 25 kg; average 8.33 kg; all days Normal

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\addep\OneDrive\Desktop\Python_ABL> & C:\Users\addep\AppData\Local\Programs\Python\Python314\python.exe c:/Users/addep/OneDrive/
Desktop/Python_ABL/programs/case42.py

Enter marks for Student 1
Enter Python marks: 80
Enter Maths marks: 85
Enter English marks: 90

Enter marks for Student 2
Enter Python marks: 70
Enter Maths marks: 75
Enter English marks: 80

Enter marks for Student 3
Enter Python marks: 90
Enter Maths marks: 95
Enter English marks: 85

--- Student Marks Report ---
Student 1
Marks: [80.0, 85.0, 90.0]
Total: 255.0
Average: 85.0
Student 2
Marks: [70.0, 75.0, 80.0]
Total: 225.0
Average: 75.0
Student 3
Marks: [90.0, 95.0, 85.0]
Total: 270.0
Average: 90.0

Top Performer: Student 3
Highest Total: 270.0
```

Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Food Waste Monitoring System program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 58: Library Borrowing and Fine Reconciliation

1. Problem Statement

Calculate library fines for borrowers based on the number of days a book is kept.

2. Objectives

Read the number of borrowers, names and borrowing days; calculate fine for overdue books; display borrower-wise details.

3. Assumptions

Allowed borrowing period is 7 days. No fine is charged for 7 days or fewer. Fine is Rs. 5 per day for each day beyond 7. These assumptions match the sample output values.

4. Algorithm

1. Start.

2. Read number of borrowers.

3. For each borrower, read name and borrowing days.

4. If days > 7, calculate fine = (days - 7) * 5; otherwise fine = 0.

5. Display name, days and fine.

6. Repeat for all borrowers.

7. Stop.

5. Flowchart

Start → Input borrower count → Read borrower name/days → Check days > 7 → Calculate fine or zero → Display borrower report → Repeat → Stop.

6. Pseudocode

```
INPUT borrower_count
FOR each borrower
    INPUT name, days
    IF days > 7
        fine = (days - 7) * 5
    ELSE
        fine = 0
    END IF
    PRINT name, days, fine
END FOR
```

7. Python Source Code

```
count = int(input("Enter number of borrowers: "))

for i in range(count):
    print()
    name = input("Enter borrower name: ")
    days = int(input("Enter number of borrowing days: "))

    if days > 7:
        fine = (days - 7) * 5
    else:
        fine = 0

    print("Borrower:", name)
    print("Borrowing days:", days)
    print("Fine: Rs.", fine)
```

8. Test Cases

Test Case 1 – Input: Jasmitha, 7 days
Expected/observed result: Fine Rs. 0

Test Case 2 – Input: Sindhura, 10 days
Expected/observed result: Fine Rs. 15

Test Case 3 – Input: Varsha, 12 days
Expected/observed result: Fine Rs. 25

Test Case 4 – Input: Deepthi, 15 days
Expected/observed result: Fine Rs. 40

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\addep\OneDrive\Desktop\Python_ABL> & C:\Users\addep\AppData\Local\Programs\Python\Python314\python.exe c:/Users/addep/OneDrive/
Desktop/Python_ABL/programs/case58.py
Enter number of borrowers: 5

Enter borrower name: jasmitha
Enter number of borrowing days: 7
Borrower: jasmitha
Borrowing days: 7
Fine: Rs. 0

Enter borrower name: thanusha
Enter number of borrowing days: 5
Borrower: thanusha
Borrowing days: 5
Fine: Rs. 0

Enter borrower name: sindhura
Enter number of borrowing days: 10
Borrower: sindhura
Borrowing days: 10
Fine: Rs. 15

Enter borrower name: varsha
Enter number of borrowing days: 12
Borrower: varsha
Borrowing days: 12
Fine: Rs. 25

Enter borrower name: deepthi
Enter number of borrowing days: 15
Borrower: deepthi
Borrowing days: 15
Fine: Rs. 40
```

Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Library Borrowing and Fine Reconciliation program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 66: Student Academic Risk Classification

1. Problem Statement

Classify a student's academic risk by analysing marks in five subjects and attendance percentage.

2. Objectives

Read student details and marks; calculate average; apply conditional rules; display the academic risk category.

3. Assumptions

High Risk if average < 40 OR attendance < 60%. Medium Risk if average < 60 OR attendance < 75%. Otherwise Low Risk. These are the rules used in this sample program.

4. Algorithm

1. Start.

2. Read student name and five subject marks.

3. Calculate average marks.

4. Read attendance percentage.

5. If average < 40 or attendance < 60, classify High Risk.

6. Else if average < 60 or attendance < 75, classify Medium Risk.

7. Otherwise classify Low Risk.

8. Display report and stop.

5. Flowchart

Start → Input name and marks → Calculate average → Input attendance → Check high-risk condition → Check medium-risk condition → Otherwise low risk → Display report → Stop.

6. Pseudocode

```
INPUT name and five subject marks
average = SUM(marks) / 5
INPUT attendance
IF average < 40 OR attendance < 60
    risk = "High Risk"
ELSE IF average < 60 OR attendance < 75
    risk = "Medium Risk"
ELSE
    risk = "Low Risk"
END IF
PRINT academic risk report
```

7. Python Source Code

```
name = input("Enter student name: ")
marks = []

for i in range(5):
    mark = float(input("Enter subject " + str(i + 1) + " marks: "))
    marks.append(mark)

attendance = float(input("Enter attendance percentage: "))
average = sum(marks) / len(marks)

if average < 40 or attendance < 60:
    risk = "High Risk"
elif average < 60 or attendance < 75:
    risk = "Medium Risk"
else:
    risk = "Low Risk"

print("\n--- Academic Risk Report ---")
print("Student:", name)
print("Marks:", marks)
print("Average:", round(average, 1))
print("Attendance:", attendance, "%")
print("Academic Risk:", risk)
```

8. Test Cases

Test Case 1 – Input: Susmitha; 90,80,85,76,80; attendance 80
Expected/observed result: Average 82.2; Low Risk

Test Case 2 – Input: Vaishnavi; 70,90,95,85,89; attendance 75
Expected/observed result: Average 85.8; Low Risk

Test Case 3 – Input: 35,40,45,30,40; attendance 80
Expected/observed result: Average 38; High Risk

Test Case 4 – Input: 55,60,58,62,60; attendance 70
Expected/observed result: Average 59; Medium Risk

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\addep\OneDrive\Desktop\Python_ABL> & C:\Users\addep\AppData\Local\Programs\Python\Python314\python.exe c:/Users/addep/OneDrive/
Desktop/Python_ABL/programs/case66.py
Enter student name: susmitha
Enter subject 1 marks: 90
Enter subject 2 marks: 80
Enter subject 3 marks: 85
Enter subject 4 marks: 76
Enter subject 5 marks: 80
Enter attendance percentage: 80

--- Academic Risk Report ---
Student: susmitha
Marks: [90.0, 80.0, 85.0, 76.0, 80.0]
Average: 82.2
Attendance: 80.0 %
Academic Risk: Low Risk
PS C:\Users\addep\OneDrive\Desktop\Python_ABL> & C:\Users\addep\AppData\Local\Programs\Python\Python314\python.exe c:/Users/addep/OneDrive/
Desktop/Python_ABL/programs/case66.py
Enter student name: vaishnavi
Enter subject 1 marks: 70
Enter subject 2 marks: 90
Enter subject 3 marks: 95
Enter subject 4 marks: 85
Enter subject 5 marks: 89
Enter attendance percentage: 75

--- Academic Risk Report ---
Student: vaishnavi
Marks: [70.0, 90.0, 95.0, 85.0, 89.0]
Average: 85.8
Attendance: 75.0 %
Academic Risk: Low Risk
PS C:\Users\addep\OneDrive\Desktop\Python_ABL>

                                        > & C:\Users\addep\AppData\Local\Programs\Python\Python314\python.exe c:/Users/addep/OneDrive/
Desktop/Python_ABL/programs/case66.py
```

Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Student Academic Risk Classification program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 71: Integrated University Fee and Eligibility Audit

1. Problem Statement

Audit a student's fee payment and attendance to determine fee clearance and academic eligibility.

2. Objectives

Read student name, amount paid and attendance; calculate remaining fee; check fee and attendance conditions; display final result.

3. Assumptions

Total fee is Rs. 55,000. Fee clearance requires payment of at least Rs. 55,000. Academic eligibility requires attendance of at least 75%. Both conditions must be met for final eligibility.

4. Algorithm

1. Start.

2. Read student name, amount paid and attendance.

3. Calculate fee balance = max(0, 55000 - amount paid).

4. Check whether amount paid >= 55000.

5. Check whether attendance >= 75%.

6. If both conditions are true, display Eligible; otherwise Not Eligible.

7. Stop.

5. Flowchart

Start → Input student/paid amount/attendance → Calculate fee balance → Check fee condition → Check attendance condition → Combine results → Display audit report → Stop.

6. Pseudocode

```
INPUT name, amount_paid, attendance
fee_balance = MAX(0, 55000 - amount_paid)
fee_ok = amount_paid >= 55000
attendance_ok = attendance >= 75
IF fee_ok AND attendance_ok
    final_result = "Eligible"
ELSE
    final_result = "Not Eligible"
END IF
PRINT fee and academic status
```

7. Python Source Code

```
name = input("Enter student name: ")
amount_paid = float(input("Enter amount paid: "))
attendance = float(input("Enter attendance percentage: "))

total_fee = 55000
balance = max(0, total_fee - amount_paid)

fee_status = "Fee Eligible" if amount_paid >= total_fee else "Fee Not Eligible"
academic_status = ("Academically Eligible" if attendance >= 75
                   else "Academically Not Eligible")
final_result = ("Eligible" if amount_paid >= total_fee and attendance >= 75
                else "Not Eligible")

print("\n--- University Audit Report ---")
print("Student:", name)
print("Total fee:", total_fee)
print("Amount paid:", amount_paid)
print("Fee balance:", balance)
print("Fee Status:", fee_status)
print("Academic Status:", academic_status)
print("Final Result:", final_result)
```

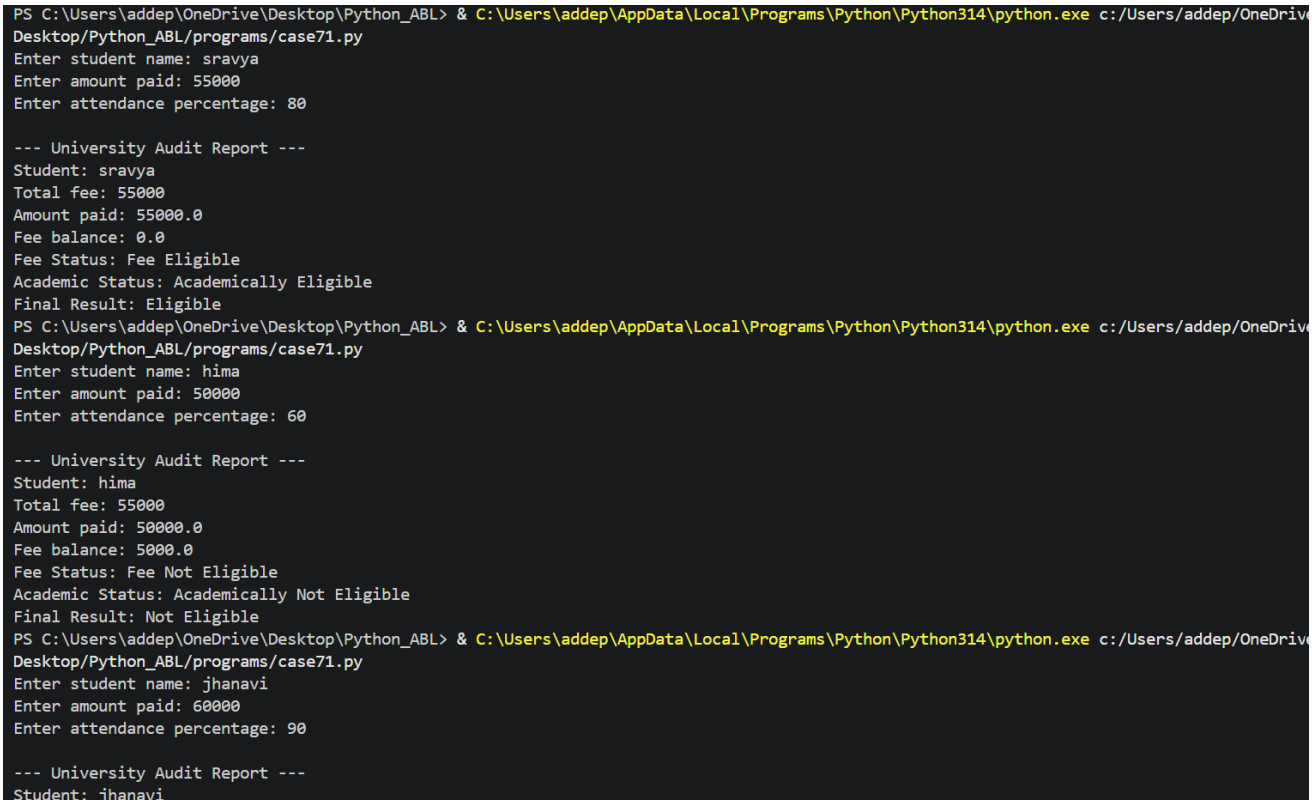
8. Test Cases

Test Case 1 – Input: Sravya; paid 55000; attendance 80
Expected/observed result: Balance 0; Fee Eligible; Academically Eligible; Eligible

Test Case 2 – Input: Hima; paid 50000; attendance 60
Expected/observed result: Balance 5000; Fee Not Eligible; Academically Not Eligible; Not Eligible

Test Case 3 – Input: Jhanavi; paid 60000; attendance 90
Expected/observed result: Balance 0; Eligible

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Integrated University Fee and Eligibility Audit program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 84: Environmental Monitoring Data Analyzer

1. Problem Statement

Analyse daily temperature, air quality index (AQI) and water quality readings to identify environmental alerts.

2. Objectives

Read environmental data for multiple days; store readings; classify each day; calculate average temperature and average AQI.

3. Assumptions

An alert is generated if temperature > 40°C OR AQI > 100 OR water quality < 50. Otherwise status is Normal. The sample run uses five days.

4. Algorithm

1. Start.
2. Read the number of days.
3. For each day, read temperature, AQI and water quality.
4. If temperature > 40 or AQI > 100 or water quality < 50, set Environmental Alert; otherwise Normal.
5. Calculate average temperature and average AQI.
6. Display each day's readings and status, followed by averages.
7. Stop.

5. Flowchart

Start → Input day count → Read temperature/AQI/water quality → Check alert thresholds → Display daily status → Calculate and display averages → Stop.

6. Pseudocode

```
INPUT number_of_days
FOR each day
    INPUT temperature, AQI, water_quality
    IF temperature > 40 OR AQI > 100 OR water_quality < 50
        status = "Environmental Alert"
    ELSE
        status = "Normal"
    END IF
    STORE readings and status
END FOR
PRINT daily reports
PRINT average temperature and average AQI
```

7. Python Source Code

```
days = int(input("Enter number of days: "))
temperatures = []
aqi_values = []
water_values = []

for i in range(days):
    print("\nEnter readings for Day", i + 1)
    temp = float(input("Temperature: "))
    aqi = float(input("AQI: "))
    water = float(input("Water quality: "))
    temperatures.append(temp)
    aqi_values.append(aqi)
    water_values.append(water)

print("\n--- Environmental Report ---")
for i in range(days):
    if temperatures[i] > 40 or aqi_values[i] > 100 or water_values[i] < 50:
        status = "Environmental Alert"
    else:
        status = "Normal"
    print("\nDay", i + 1)
    print("Temperature:", temperatures[i])
    print("AQI:", aqi_values[i])
```

```
    print("Water Quality:", water_values[i])
    print("Status:", status)

if days > 0:
    print("\nAverage Temperature:", round(sum(temperatures) / days, 1))
    print("Average AQI:", round(sum(aqi_values) / days, 1))
```

8. Test Cases

Test Case 1 – Input: 30,60,80; 42,60,80; 30,120,80; 30,60,40; 32,50,80
Expected/observed result: Alerts on days 2, 3 and 4; average temperature 32.8; average AQI 70

Test Case 2 – Input: 25,40,75
Expected/observed result: Normal; average temperature 25; average AQI 40

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW



Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Environmental Monitoring Data Analyzer program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.

Case 100: Comprehensive Smart Campus Management Simulation

1. Problem Statement

Develop a simple campus management program that records student attendance, fee payment, library fine and campus event registration, then displays each student's status.

2. Objectives

Accept details for multiple students; check academic attendance and fee payment; check library fine; record event registration; print individual reports and a campus summary.

3. Assumptions

Total fee is Rs. 55,000; attendance eligibility is at least 75%; fee must be paid in full; library fine must be zero for the combined eligibility status. Event registration is reported separately. These conditions follow the supplied sample output.

4. Algorithm

1. Start.
2. Read the number of students.
3. For each student, read name, attendance, fee paid, library fine and event registration choice.
4. Check attendance >= 75, fee paid >= 55000 and library fine == 0.
5. Display Eligible only if all three conditions are satisfied.
6. Display event registration status separately.
7. Repeat for all students and print campus summary.
8. Stop.

5. Flowchart

Start → Input student count → Read each student's details → Check attendance, fee and library fine → Determine combined eligibility → Display event status → Repeat → Campus summary → Stop.

6. Pseudocode

```
INPUT number_of_students
FOR each student
    INPUT name, attendance, fee_paid, library_fine, event_choice
    IF attendance >= 75 AND fee_paid >= 55000 AND library_fine == 0
        status = "Eligible"
    ELSE
        status = "Not Eligible"
    END IF
    IF event_choice is yes: event_status = "Registered"
    ELSE: event_status = "Not Registered"
    PRINT student report
END FOR
PRINT campus summary
```

7. Python Source Code

```
count = int(input("Enter number of students: "))
total_students = count
eligible_count = 0

for i in range(count):
    print("\nEnter details for Student", i + 1)
    name = input("Student name: ")
    attendance = float(input("Attendance percentage: "))
    fee_paid = float(input("Fee amount paid: "))
    library_fine = float(input("Library fine: "))
    event = input("Register for campus event? (yes/no): ").lower()

    if attendance >= 75 and fee_paid >= 55000 and library_fine == 0:
        status = "Eligible"
        eligible_count += 1
    else:
```

```
        status = "Not Eligible"

    event_status = "Registered" if event == "yes" else "Not Registered"

    print("\n--- SMART CAMPUS MANAGEMENT REPORT ---")
    print("Student:", name)
    print("Attendance:", attendance, "%")
    print("Fee Paid: Rs.", fee_paid)
    print("Library Fine: Rs.", library_fine)
    print("Academic and Fee Status:", status)
    print("Event Status:", event_status)

print("\n--- CAMPUS SUMMARY ---")
print("Total Students:", total_students)
print("Eligible Students:", eligible_count)
print("Not Eligible Students:", total_students - eligible_count)
```

8. Test Cases

Test Case 1 – Input: Jasmitha; attendance 85; fee 55000; fine 0; event yes
Expected/observed result: Eligible; Event Status: Registered

Test Case 2 – Input: Thanusha; attendance 90; fee 50000; fine 100; event no
Expected/observed result: Not Eligible; Event Status: Not Registered

9. Output Screenshot(s) — INSERT YOUR OWN SCREENSHOT BELOW

```
PS C:\Users\addep\OneDrive\Desktop\Python_ABL> & C:\Users\addep\AppData\Local\Programs\Python\Python314\python.exe c:/Users/addep/OneDrive/
Desktop/Python_ABL/programs/case100.py
Enter number of students: 2

Enter details for Student 1
Student name: jasmitha
Attendance percentage: 85
Fee amount paid: 55000
Library fine: 0
Register for campus event? (yes/no): yes

Enter details for Student 2
Student name: thanusha
Attendance percentage: 90
Fee amount paid: 50000
Library fine: 100
Register for campus event? (yes/no): no

--- SMART CAMPUS MANAGEMENT REPORT ---

Student: jasmitha
Attendance: 85.0 %
Fee Paid: Rs. 55000.0
Library Fine: Rs. 0.0
Academic and Fee Status: Eligible
Event Status: Registered

Student: thanusha
Attendance: 90.0 %
Fee Paid: Rs. 50000.0
Library Fine: Rs. 100.0
Academic and Fee Status: Not Eligible
Event Status: Not Registered

--- CAMPUS SUMMARY ---
Total Students: 2
```

Supplied terminal screenshot. Replace/add your final complete run screenshot if required.

10. Conclusion

The Comprehensive Smart Campus Management Simulation program was developed using basic Python concepts such as variables, input/output, arithmetic operators, conditional statements, loops and lists. It processes the given inputs and displays the required report. The test cases help verify the program's main conditions.